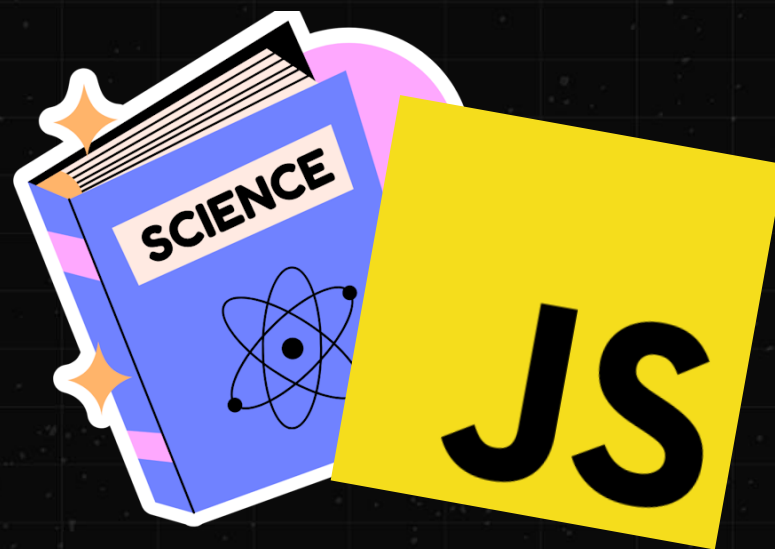


ЛЕКЦИЯ #4

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Будем говорить, как преодолеть ограничение на работу с байтом и **работать с отдельными битами или группой бит**
- * Научимся **работать с побитовыми операторами в JS**
- * Поговорим про **бинарные форматы данных** и сравним их с текстовыми
- * Поговорим про **другие кейсы**, где применяют побитовые операторы

ПРОЦЕССОР РАБОТАЕТ С БАЙТАМИ

- * Для него это **минимальный «атом» информации**
- * Проблема в том, что бывают ситуации, когда **использовать только байты слишком расточительно**
- * Например, **битовый вектор будет в 8 раз эффективнее** обычного вектора булевых значений!

БИНАРНЫЕ ФОРМАТЫ ДАННЫХ

- * Особенностью многих бинарных форматов данных являются **специальные заголовки**
- * В них хранится некоторая **метаинформация о данных**
- * Например, версия используемого формата или хеш-сумма для данных
- * Часто, это небольшая информация, для кодирования которой **хватило бы всего несколько бит**
- * Вместо того, чтобы каждому заголовку выделять отдельные байты – их **можно упаковать внутри одного или нескольких байт**

Что означают биты

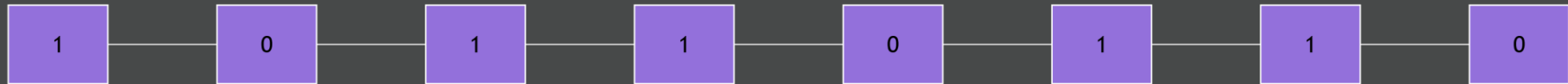
● Номер версии (биты 0-2)
 $110_2 = 6 \rightarrow$ версия 6

● Тип данных (биты 3-4)
 $00_2 = 0 \rightarrow$ тип: целое
число

● Приоритет (биты 5-7)
 $101_2 = 5 \rightarrow$ высокий
приоритет

● Размер данных (биты 8-15)
 $10110010_2 = 178$ байт

Байт 1



Байт 0



ИЛИ ЕЩЕ ПРОЩЕ

- * Чтобы узнать, что число отрицательное, можно посмотреть на значение **крайне левого бита**
- * Если там **1**, то значит число отрицательно
- * Осталось понять как это сделать

ДЛЯ ЭТОГО И
ИСПОЛЬЗУЮТСЯ
ПОБИТОВЫЕ
ОПЕРАТОРЫ?



ДА

- * Хотя кейсы их использования куда шире
- * В конце концов, любое «сложное» вычисление строится из **комбинации побитовых операций**
- * Но с прикладной точки зрения, **чтение отдельных бит в рамках байта** – наиболее частая задача для побитовых операторов

УЗНАЕМ ЗНАК ЧИСЛА

```
function isNegative(num) {  
    return (num & (1 << 31)) !== 0;  
}  
  
console.log(isNegative(42)); // false  
console.log(isNegative(-42)); // true
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВСЕ НЕ ТАК СТРАШНО

- * Простейший способ разобраться с побитовыми операторами – это **визуализировать операции в двоичном виде**
- * Давайте **распишем по шагам** что происходит в функции определения знака
- * Держите в голове, что применение побитовых операторов **преобразует наше число в 32 битное целое**

```

function toBinaryString(num) {
  const data = new Int32Array([num]);
  const bytes = new Uint8Array(data.buffer);

  let str = "";

  for (let i = bytes.length; i--;) {
    const byte = bytes[i].toString(2).padStart(8, "0");
    str += byte + (i === 0 ? "" : "_");
  }

  return str;
}

function isNegative(num) {
  console.log("На вход подали число:\n");
  console.log(toBinaryString(num));

  console.log("Создадим маску на основе числа 1:\n");
  console.log(toBinaryString(1));

  console.log("Применим к 1 операцию << 31:\n");
  console.log(toBinaryString(1 << 31));

  console.log("Применим к числу нашу маску num & maskL:\n");
  console.log(toBinaryString(num & (1 << 31)));

  return (num & (1 << 31)) !== 0;
}

console.log(isNegative(42));
console.log(isNegative(-42));

```

На вход подали число:
00000000_00000000_00000000_00101010

Создадим маску на основе числа 1:
00000000_00000000_00000000_00000001

Применим к 1 операцию << 31:
10000000_00000000_00000000_00000000

Применим к числу нашу маску num & maskL:
00000000_00000000_00000000_00000000

Это 0, поэтому 0 !== 0 дает false

На вход подали число:
11111111_11111111_11111111_11010110

Создадим маску на основе числа 1:
00000000_00000000_00000000_00000001

Применим к 1 операцию << 31:
10000000_00000000_00000000_00000000

Применим к числу нашу маску num & maskL:
10000000_00000000_00000000_00000000

Это не 0, поэтому 0 !== 0 дает true

ПОБИТОВОЕ «И» &

✱ Дает 1, только если оба операнды 1

✱ $0 \& 0 == 0$ | $1 \& 0 == 0$ | $0 \& 1 == 0$ | $1 \& 1 == 1$

НО ПОЧЕМУ
5 & 3 == 1?



ТАКИЕ ОПЕРАТОРЫ РАБОТАЮТ «ПОБИТОВО»

- * Значение переводится в формат целого 32 битного числа и к каждому биту числа применяется операция

- * Результат тоже является другим числом

- * Запишем 5 и 3 в двоичной системе и сделаем &

- * 0b0101

&

0b0011

==

0b0001

ТЫ ЖЕ ГОВОРИЛ,
ЧТО ПРОЦЕССОР
НЕ УМЕЕТ
РАБОТАТЬ С
БИТАМИ?



ТАК И ЕСТЬ

- * Операция `&` и другие **работают на уровне байт**
- * Но нас появилась **возможность «отбрасывать» лишние биты**, чтобы работать только с нужными
- * Допустим мы хотим прочитать **биты с 24 по 32** в некотором 32 битном числе
- * Нам нужно сделать число, где **в нужных позициях стоит 1, а в остальных 0**
- * Такое число называется **маской**
- * **Применение маски к исходному числу** через `&` «отбросит» все лишние биты

КАК СОЗДАТЬ МАСКУ?



ЭТО НЕ СЛОЖНО

```
function createMask(position, size) {  
  let mask = 1;  
  mask = (mask << size) - 1;  
  mask = mask << position;  
  return mask;  
}
```


second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ 😊

- * Разберем по шагам, что происходит
- * Первое, что нужно сделать – это число, где **ровное количество 1 (размер)**, сколько требует маска
- * Возьмем за основу **1** (`0b0000000000000001`) и сдвинем все биты этого числа **влево на размер нашей маски** (в нашем случае это **8**)
- * Получится число `0b0000000100000000` (**256**)
- * Отнимаем **1** и получается `0b0000000011111111`
- * Ровно то, что нам и нужно!

ЕСЛИ НЕ ПОНЯЛИ

- * Мы воспользовались свойством **позиционных систем счисления**
- * Как сделать число **999** в десятичной системе: берем **1000** и отнимаем **1**
- * Тоже самое мы делаем и в двоичной системе: сдвигая **1** влево на заданное количество бит мы **делаем число на 1 больше нужного нам**
- * Отнимаем **1** и получаем нужное

ТЕПЕРЬ НУЖНО СДВИНУТЬ МАСКУ

* Мы хотели считывать биты начиная с 24, поэтому просто сдвигаем маску на это число влево

* `0b00000000000000000000000011111111 << 24`

`==`

`0b11111111000000000000000000000000`

* Вот и все, теперь маска готова к использованию

АРИФМЕТИЧЕСКИЙ СДВИГ ВЛЕВО <<

- * Все биты числа **сдвигаются** на заданное количество бит влево
- * Справа «**дозаписываются**» нули
- * `0b0101 << 2 == 0b010100`
- * С математической точки зрения «эффект» от такой операции **равен умножению на степень двойки**
- * В данном случае на **4** (**2** в степени **2**) и равно **20** (`0b0101` – это **5**)
- * Поэтому сдвиг и **называется арифметическим**

А ЧТО
ПРОИСХОДИТ
С КРАЙНЕ
ЛЕВЫМИ
БИТАМИ?



ПРОИСХОДИТ ПЕРЕПОЛНЕНИЕ

* Все, что находится слева – просто отбрасывается

* `0b11111111000000000000000000000000 << 2`

`==`

`0b11111100000000000000000000000000`

ОПТИМИЗАЦИЯ УМНОЖЕНИЯ И ДЕЛЕНИЯ

- * Помните я говорил, что при разбиение на разряды целого числа деление на 10 является тяжелой операцией?
- * Все дело в том, что деление – это многократное повторение вычитания
- * А умножение – многократное сложение
- * Но когда у нас степень двойки, то умножать и делить становится очень просто
- * Разумеется, речь про целочисленные операции
- * Большинство компиляторов и виртуальных машин оптимизируют такие операции через сдвиги



```
const list1 = new Array(15000);
const list2 = new Array(16384); // Степень двойки

function set(i, item, list) {
  // Если list1.length степень двойки, то будет оптимизация деления
  list[i % list.length] = item;
}

// Прогрев
for (let i = 0; i < 1e9; i++) {
  set(i, i, list1);
}

console.time("Не степень двойки");

for (let i = 0; i < 1e9; i++) {
  set(i, i, list1)
}

console.timeEnd("Не степень двойки");

console.time("Степень двойки");

for (let i = 0; i < 1e9; i++) {
  set(i, i, list2)
}

console.timeEnd("Степень двойки");
```



```
node --jitless ./bench.js
# Не степень двойки: 17.563s
# Степень двойки: 17.495s
```



```
node ./bench.js
# Не степень двойки: 1.063s
# Степень двойки: 537.639ms
```

ВЫВОДЫ

- * JIT компилятор оптимизировал деление на степень двойки и на примере V8 это дало двухкратный прирост производительности
- * Разница С и БЕЗ JIT просто огромна!
- * Самостоятельно **делать такие оптимизации тоже можно**, но зачастую – бессмысленно (эффект только без JIT)
- * Разумеется, в **других VM результат может отличаться**, но на практике – большинство компиляторов и VM используют такие оптимизации

А КАК
ДЕЛИТЬ НА
СТЕПЕНЬ 2?



АРИФМЕТИЧЕСКИЙ СДВИГ ВПРАВО >>

- * Все биты числа **сдвигаются** на заданное количество бит вправо
- * Слева «**дозаписываются**» значения битового знака (знак не меняется)
- * `0b0101 >> 2 == 0b0001`
- * С математической точки зрения «эффект» от такой операции **равен деление на степень двойки**
- * В данном случае на **4** (**2** в степени **2**) и равно **1** (`0b0101` – это **5**)
- * **Деление целочисленное**, поэтому результат **1**

ЛОГИЧЕСКИЙ СДВИГ ВПРАВО >>>

- * Все биты числа сдвигаются на заданное количество бит вправо
- * Слева «дозаписываются» нули (число всегда становится положительным)
- * $-1 \ggg 0 == 4294967295$
- * Для положительных чисел эффект такой же, как и при $\gg$

ЗАЧЕМ **НУЖЕН**
ЭТОТ СДВИГ?



ДЛЯ НАС ЭТОТ СДВИГ ДАЖЕ ПОЛЕЗНЕЕ

- * Он позволяет **просто сдвинуть биты «как есть»** без логики знака (прямой код, а не дополнительный)
- * Например, я хочу прочесть некоторые 4 бита в числе, как цифру в BCD
- * // Читаем вторую справа цифру и маской отсекаем лишние биты
(bcd >>> 4) & 0b1111

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ, РАСПИШЕМ ПО ШАГАМ

- * Допустим у нас число **971** в BCD 8421 будет выглядеть как **0b1001_0111_0001**
(в прямом коде это будет число **2417**)
- * И мы хотим прочитать вторую цифру справа (**0111**)
- * Даже если применить маску и сделать число **0b0000_0111_0000**,
то возникают **проблемы с интерпретацией этого числа**
- * Мы знаем, что это число **7**, но в отладчике мы увидим **112**
(из-за **0000** справа)
- * А сдвигая вправо мы избавляем от них и получаем **7**
(в BCD 8421 цифры кодируются прямым кодом)

МОЖНО НЕ ДВИГАТЬ МАСКУ

- * Когда мы просто читаем значение по некоторой маске, то можно **не двигать маску на нужную позицию, а сдвинуть само значение**
- * Именно это мы тут и делаем `(bcd >>> 4) & 0b1111`

А КАК ЗАКОДИРОВАТЬ ЗНАЧЕНИЕ?



НАПРИМЕР, ВСД ВНУТРИ ЧИСЛА

- ✱ Нужно просто сделать такие числа, где все биты стоят на своих позициях
- ✱ А затем объединить их в одно число с помощью побитового «или» |
- ✱ Например, $9 \ll 8 \mid 7 \ll 4 \mid 1$

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЖКА, СТОЖКААА!

ОЙ, ДА ЛАДНО, САМОЕ СЛОЖНОЕ ПОЗАДИ

- ✱ Мы уже освоили работу почти со всеми побитовыми операторами в JS
- ✱ Крепитесь, осталось чуть-чуть

ПОБИТОВОЕ «ИЛИ» |

* Дает 1, если хотя бы один операнд 1

* $0 | 0 == 0$ | $1 | 0 == 1$ | $0 | 1 == 1$ | $1 | 1 == 1$

РАСПИШЕМ ПО ШАГАМ

* $9 \ll 8 == 0b1001_0000_0000$
 $7 \ll 4 == 0b0000_0111_0000$
 $1 == 0b0000_0000_0001$
 |
 $0b1001_0111_0001$

* Получилось число, **внутри которого закодированы 3 BCD цифры**

БИТОВЫЕ МАСКИ ДЛЯ ПРИЗНАКОВ

```
// Мы можем закодировать некоторые признаки в виде чисел
const CAN_READ = 0b0000_0001;
const CAN_WRITE = 0b0000_0010;
const CAN_DELETE = 0b0000_0100;

// Теперь опишем роль, которая может читать и писать
const editor = CAN_READ | CAN_WRITE;

// Для проверки прав используем &
if (role & CAN_READ) {
    // Данная роль умеет читать
}
```


А В ЧЕМ ПРОФИТ?



ЭКОНОМИЯ ПАМЯТИ И ЭФФЕКТИВНОСТЬ

- * Обычно мы храним такие признаки в виде объекта:
`{canRead: true, canWrite: true, canDelete: false}`
- * Объект – это сложная структура данных в куче, которая занимает **сильно больше памяти, чем одно число**
- * Особенно, если число храниться как SMI
- * Это **сильно эффективнее по памяти и быстрее работает**
- * Многие библиотеки, например, Vue, использует этот подход

БИТОВЫЕ МАСКИ ИСПОЛЬЗУЮТСЯ ПОВСЕМЕСТНО

- * Например, наше устройство в сети имеет IP адрес
- * Часть адреса – это адрес устройства, а остальная – адрес подсети
- * Адрес подсети **задаётся маской подсети**

IP-адрес:	11000000 10101000 00000000 1 00000010	(192.168.1.2)
Маска подсети:	11111111 11111111 11111111 0 00000000	(255.255.254.0)
Адрес сети:	11000000 10101000 00000000 0 00000000	(192.168.0.0)

ПОБИТОВОЕ «НЕ» ~

- * Дает 1, если значение 0 и 0 – если 1
- * `~0 == 1` | `~1 == 0`
- * Можно **сменить знак числа**, например, `~42 + 1 == -42`
- * Иногда используют с методами, которые возвращают `-1`, ведь `~-1 == 0`
- * Например, `if (~[1, 2, 3].indexOf(10)) { console.log("Не найдено!"); }`
- * Применение `~0` даст число из одних единиц (`-1` или `4294967295`)

ПОБИТОВОЕ «ИСКЛЮЧАЮЩЕЕ ИЛИ» $\wedge$

- * Дает 1, только если один из операндов операнд 1
- * $0 \wedge 0 == 0$ | $1 \wedge 0 == 1$ | $0 \wedge 1 == 1$ | $1 \wedge 1 == 0$
- * Очень жирно используется в криптографии и в различных алгоритмах
- * $3434534 \wedge 232 == 3434702$
 $3434702 \wedge 232 == 3434534$

НАПРИМЕР, СУММАТОР

```
function add(a, b) {  
  while (b !== 0) {  
    const sum = a ^ b;  
    const carry = (a & b) << 1;  
    a = sum;  
    b = carry;  
  }  
  
  return a;  
}  
  
console.log(add(5, 17)); // 22
```

СЛОЖНОЕ ДЕЛАЕТСЯ ИЗ ПРОСТОГО

- * Такое сложное устройство как процессор строится из простых «логических вентилях»
- * Это и есть наши побитовые операторы и они лежат в основе всего, что умеет наш компьютер
- * Более того, если вы добавите новые числовые типы (BCD, BigFloat и другие), то вам придется самостоятельно реализовывать для них арифметику

ГЕНЕРАТОР ПСЕВДОСЛУЧАЙНЫХ ЗНАЧЕНИЙ

```
class XorShift {  
  constructor(seed = Date.now()) {  
    this._seed = seed  
  }  
  
  random() {  
    this._seed ^= this._seed << 13  
    this._seed ^= this._seed >> 17  
    this._seed ^= this._seed << 5  
    return this._seed < 0 ? 1.0 + (this._seed / 0x80000000) : this._seed / 0x7FFFFFFF;  
  }  
}
```


А ЗАЧЕМ ЭТО
НУЖНО?



НАПРИМЕР, ДЛЯ SSR

- * В SSR важно, чтобы **генерируемый сервером ответ совпадал** с тем, что воссоздается (гидратируется) на клиенте
- * С *Math.random()* будут проблемы



```
const rand1 = new XorShift(42);  
const rand2 = new XorShift(42);  
  
console.log(rand1.random() === rand2.random()); // true  
console.log(rand1.random() === rand2.random()); // true
```

РЕФЛЕКСИРУЕМ

- * Побитовые операторы не только позволяют интерпретировать биты внутри байта, но и **являются сердцем огромного числа алгоритмов**
- * Да и **сердцем самих вычислений**

ПОМНИМ ПРО JS СПЕЦИФИКУ

- * Применение побитовых операторов к `number` преобразует внутреннее представление в формат 32 битного целого числа
- * Число больше 32 бит будет переполняться (`2 ** 32 >>> 0 == 0`)
- * Логический сдвиг вправо `>>>` всегда делает число беззнаковым
- * `-1 >> 0 == 4294967295` (число в памяти не поменялось – поменялась интерпретация)
- * Остальные побитовые операторы учитывают знаковый бит

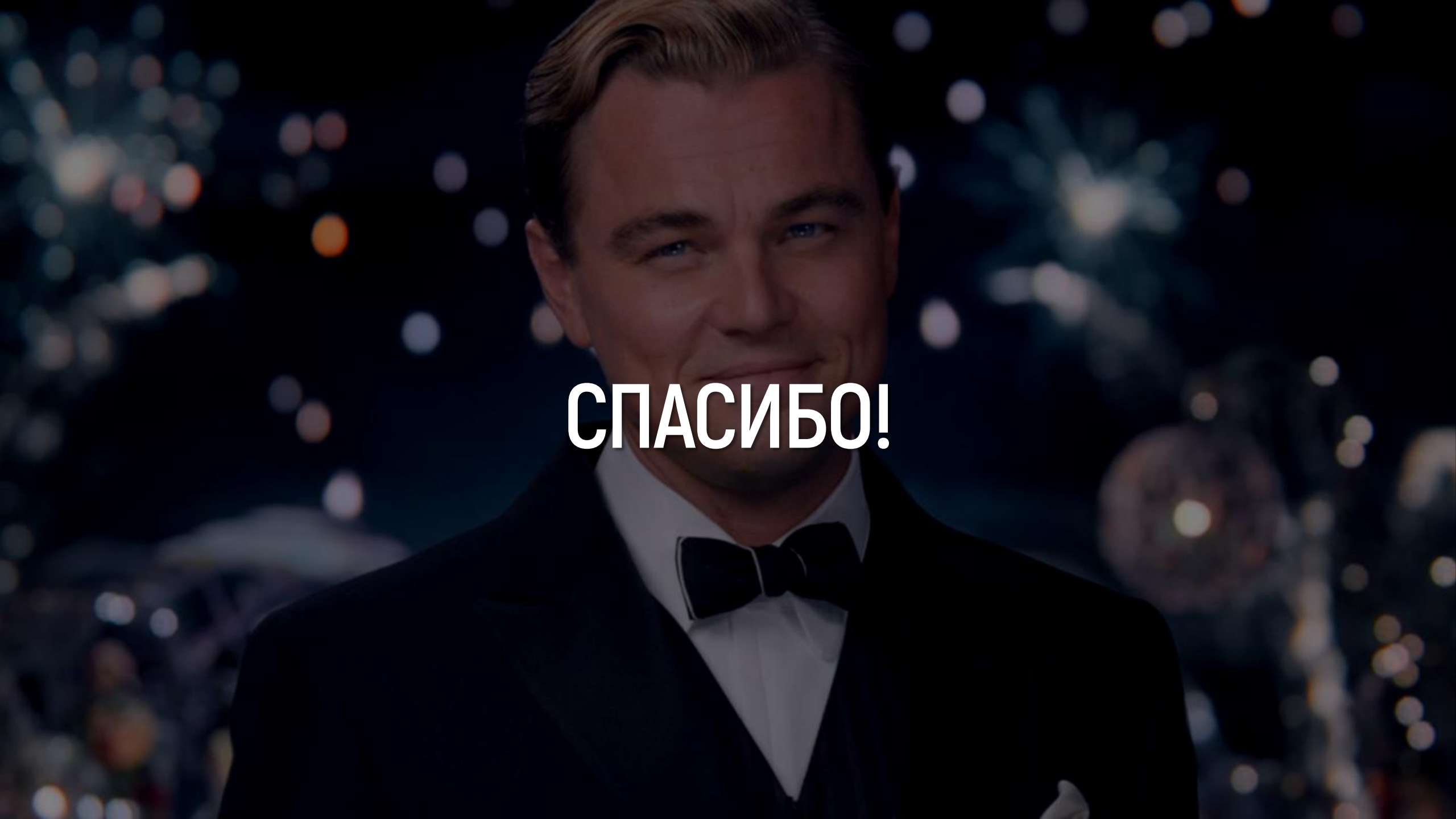
С **bigint** ВСЕ ПРОЩЕ

- * Ничего никуда не преобразуется – **bigint** остается **bigint**
- * Логический сдвиг вправо **bigint** просто не поддерживает
- * Оба операнда должны быть **bigint**

```
console.log(2n << 2n); // 8n
console.log(8n >> 2n); // 2n
console.log(8n >>> 2n); // TypeError: BigInts have no unsigned right shift, use >> instead
```

ПОДВЕДЕМ ИТОГИ

- * Побитовые операторы – это операторы которые **работают со значениями побитово**
- * В JS они **поддерживаются только числовыми типами**
- * Тип **number** всегда приводится к Int32/Uint32
- * Важнейшим понятием при работе с битовыми операторами является маска
- * **Нужна практика!**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!